

Explain how the congestion control mechanisms were implemented (slow start, congestion avoidance, fast retransmit).

All the control mechanisms are implemented in the client in a try/except call. The slow start multiplies the cwnd by 2 with every acknowledgement that comes back from the server to get exponential growth, like the TCP congestion control. When the congestion control gets into the congestion avoidance phase, the cwnd is calculated by adding 1, which results in the linear growth. In order to keep track of the duplicate acknowledgments that come through, I implemented a dictionary that has the acknowledgment number and the number of retransmissions. If the value gets to 3, then the ssthresh is cwnd count in half, the cwnd is also cut in half, and then resend the packet once again. If there is a dup that is not a triple dup, the client sends the whole cwnd again not just the one packet that is getting the dup.

Describe how timeouts and retransmissions were handled.

The timeouts occur in the except part of the try/except call. The except is triggered by the socket timeout which is set to 500ms. If it is triggered, the ssthresh is half of the cwnd, the cwnd is 1, and the packet is retransmitted to the server. To track the retransmissions for the packets, I keep track of the retransmissions and I increase the variable whenever a timeout or retransmission happens. When all packets are sent for that batch, I save the cwnd and the retransmission rate for that round to be graphed later.

Provide insights from the graphs and discuss any patterns or observations (e.g., how packet loss affected cwnd, RTT variations).

The best looking and most readable graphs come from the 1% packet loss, as there wasn't that much packet loss. The manually induced timeout clearly shows the slow start and congestion avoidance stages. The cwnd and retransmission graphs seem to be accurate because the retransmissions increase around the same time the cwnd changed, which is a good sign. The 10% is pretty similar to the 1% as no timeouts minus the manually induced timeout should be happening. The 50% graph is a little wacky looking because there can be timeouts so it is a lot of zig zaging. Overall the graph's behaviors are expected. A pattern that indicates that these graphs are correct is that when the cwnd grows high, the retransmissions are zero during that time.

Discuss any challenges faced and lessons learned during the implementation.

There were a lot of challenges when implementing this program. Initially, I thought I had the assignment done, but after going to the lab, I realized that I was getting way too many timeouts and the packets weren't sending correctly and the acks weren't sending correctly. It took me about two days with the help of going to office hours and slacking Dev for me to finally understand the assignment and what needed to happen. Originally, I didn't implement the server correctly because I forgot about the RTT being 100ms. Once I implemented that the batching started to work on the client where the server would wait to receive more packets until after the

ack was sent. Overall, this assignment was much harder than it needed to be. I think for the next time this assignment is used, more detailed instructions on what needs to be done would've been nice.